ICS Lab 2

1. 基本方法

本实验采用以下分析流程：

1) 使用“ objdump -d --no-show-raw-insn --disassemble=函数名 ./bomb > 函数名.S ”导出指定函数的汇编，入口函数调用的辅助函数也要一并导出。

2) 观察函数参数进入哪些寄存器，以及相关寄存器如何经过移位、异或、加法、乘法等运算。

3) 分析 cmp、test 和条件跳转指令，判断输入需要满足的条件。

4) 将汇编指令按执行顺序还原为数学表达式或递归关系。

5) 在 defuser.c 中实现求解函数，并通过不同 seed 的测试验证。

验证流程对六关通用：先固定 seed 42，用占位口令运行炸弹，在关键比较指令处设断点，把寄存器中的期望值与按恢复公式计算的结果核对。一致后将公式写入 defuser.c 并重新编译，运行 python3 public_test.py --phase N 公开测试。最后用一个公开列表以外的 seed（本报告取 seed 1205，记录见第 9 节）复核公式对任意 challenge 成立。后文各关只记述本关特有的核对内容。

2. Phase 1

2.1 关键汇编分析

phase_01 以 esi = 1 调用 valid_u32_fields 确认口令是 1 个无符号整数，sscanf 解析到 -0x14(%rbp) 后调用 p1_compute，返回值与输入整数比较。全部计算集中在 p1_compute 及其调用的三个子函数中：

1) p1_compute 用 shr $0x20、shr $0x10 与 mov 的 32 位别名，把 challenge 拆成高 32 位、中段和低 32 位三份暂存。

2) p1_step0、p1_step1、p1_step2 分别实现三条计算链；乘 5 均以 lea 比例变址完成，没有出现 imul。

3) 立即数 0x802b、0x8496、0xd8b7 分别锚定三条链，是定位计算关系的突破口。

4) p1_compute 按调用顺序串联三者，末尾把 p1_step0 与 p1_step2 的结果异或后返回。

2.2 恢复出的计算关系

令输入 challenge 为 64 位无符号整数 C，定义：

$$x = C \bmod 2^{32}$$

$$h = \left\lfloor \frac{C}{2^{32}} \right\rfloor,\ m = \left\lfloor \frac{C}{2^{16}} \right\rfloor \bmod 2^{32}$$

根据指令的执行顺序，构造中间变量：

$$a = (h + 0x802b)\ \hat{}\ x$$

$$b = (5h + m)\ \hat{}\ 0x8496$$

$$c = 5(b - x)\ \hat{}\ 0xd8b7$$

最后，输出的答案为：

$$v_0 = c\ \hat{}\ a$$

以上中间计算均按 32 位无符号整数运算解释。

2.3 推理过程

首先，确定口令形式：口令是 1 个整数，且必须等于 p1_compute 的返回值。

其次，分析 challenge 的拆分方式。三处取值分别对应低 32 位的 x、高 32 位的 h，以及 (C>>16) 的低 32 位 m。其中，m 对应 C 的第 16 至 47 位，与 x 和 h 都有部分重叠。这是因为右移 16 位后没有再去掉高 16 位，而 32 位寄存器只保留结果的低 32 位。若误以为这三部分互不重叠，就会导致后续计算出错，调试时也会发现 a 的值首先不一致。

三条计算链分别对应三个小函数，可根据指令中的常数判断其作用：(h+0x802b) ^ x 得到 a，(5h+m) ^ 0x8496 得到 b，5(b-x)+0xd8b7 得到 c。p1_compute 按顺序调用它们，最终返回 c ^ a。由于 c 依赖 b，计算顺序不能随意调整。

实现时统一使用 uint32_t，使乘法和加法的结果超出 32 位时自动按模 (2^{32}) 回绕，避免因使用有符号类型或 64 位中间结果而产生错误。

3. Phase 2

3.1 关键汇编分析

首先，根据入口处 esi=6 调用 valid_u32_fields，以及 sscanf 的返回值必须为 6，可确定口令由 6 个整数构成。6 个整数依次存放在栈上从 -0x40(%rbp) 开始的数组中，每个元素占 4 字节。

关键指令的分析如下：

1) 数组访问： -0x40(%rbp,%rdx,4) 表示从数组起始位置向后移动 rdx × 4 字节，说明程序按 4 字节读取整数。

2) 第一个整数： 程序先单独检查第一个整数，计算 (challenge >> 16) & 0x1f，再加 1。因此，第一个整数必须等于该结果。

3) 后五个整数： 循环中，cl 从 5 开始，每轮增加 4，用于控制右移位数；右移后通过 and $0x1f 取低 5 位，再加上当前下标 rdx。同时，lea (%rsi,%rsi,2),%esi 将前一项乘以 3，随后与前面的计算结果异或，得到下一项。由此可还原后五个整数的递推关系。

4) 循环范围： rdx 从 1 递增到 5，当 rdx 达到 6 时退出循环，说明循环依次检查第 2 至第 6 个整数。

因此，Phase 2 的关键是先根据位移和掩码确定第一个整数，再按照循环中的乘法、异或和下标运算，依次计算其余五个整数。

3.2 恢复出的计算关系

初始值为：

$$v_0 = ((C \gg 16)\ \&\ 0x1f) + 1$$

对于 $i = 1,2,\ldots,5$，定义：

$$b_i = (C \gg (5 + 4(i-1)))\ \&\ 0x1f$$

根据循环中各条运算指令，得到递推式：

$$v_i = (b_i + i)\ \hat{}\ (3v_{i-1})$$

最终答案包含六个整数：

$$(v_0, v_1, v_2, v_3, v_4, v_5)$$

3.3 推理过程

第一个整数在循环外单独检查，使用固定右移 16 位；后五个整数则在循环中计算，右移位数从 5 开始，每轮增加 4。这说明第一个整数与后续项的计算方式不同。具体来说，第一个整数取 challenge 第 16–20 位的值；后五项依次取第 5–9、9–13、13–17、17–21、21–25 位的值。由于这些位段存在重叠，不能把它们当作互不相关的部分，而应按照各自的移位和掩码操作分别取值。

此外，lea (%rsi,%rsi,2), %esi 实现前一项乘以 3。递推时必须按下标从 1 到 5 依次计算，因为每一项都依赖前一项，不能跳过中间步骤，也不能简单套用等差或等比数列的规律。

4. Phase 3

4.1 关键汇编分析

phase_03 先以 esi=2 检查并读取两个整数。随后，程序计算 challenge & 7，并与第一个整数比较，因此第一个整数必须等于 challenge 的最低 3 位所表示的数。

接着，程序通过右移 3 位、右移 32 位并进行异或运算，计算出操作数 x，再将其传入 p3_dispatch。函数返回值随后与第二个整数比较，说明第二个整数必须等于对应分支的计算结果。

p3_dispatch 使用跳转表选择分支：程序先检查编号是否大于 7，再以该编号为下标，从地址 0x405820 处的表中读取目标地址并跳转。由于编号由 challenge & 7 得到，范围只能是 0–7，因此大于 7 的检查在正常情况下不会触发。八个分支分别执行不同的计算，主要使用 lea、add、xor 和 shr 等指令，没有使用 imul。因此，可以根据这些指令逐一还原各分支的运算公式。

4.2 恢复出的计算关系

分支编号为：

$$j = C \,\&\, 7$$

定义：

$$x = (C \gg 3) \text{ ^ } (C \gg 32)$$

其中参与后续运算的部分按 32 位无符号整数解释。

根据分支编号 j，结果 r 为：

$$r = \begin{cases} (4x) \text{ ^ } 0xf50c, & j = 0 \\ 5x + 0x1e555, & j = 1 \\ 5x + 0x33ac2, & j = 2 \\ 3x + 0x90bb, & j = 3 \\ 5x + 0xaaa6, & j = 4 \\ (x \text{ ^ } 0x7e81) + (x \gg 1), & j = 5 \\ (x \text{ ^ } 0x4b8a) + (x \gg 1), & j = 6 \\ (2x) \text{ ^ } 0x813d, & j = 7 \end{cases}$$

答案为两个整数：

$$(j, r)$$

4.3 推理过程

首先，程序在计算前检查第一个整数是否等于 challenge & 7，因此可以确定分支编号由 challenge 的最低 3 位决定。该编号在调用 p3_dispatch 前就已算

好，并通过寄存器传入，说明八个分支使用同一个编号，区别在于各自的计算方式。

接着，逐一追踪跳转表中的地址：表项先跳到 p3_dispatch 内的跳转代码，再由其跳转到对应的分支函数。只有将这三者逐一对应，才能确定分支编号与函数的真实关系。需要注意，符号表中的 p3_case_N 按函数地址命名，并不一定对应编号为 N 的分支，不能直接按函数名判断。这是本关最容易出错的地方。核对后发现，八个分支的运算主要由乘以 2、3、4 或 5，再结合常数加法、异或等操作构成，另有一种混合运算。其中，j=5 和 j=6 的结构相同，只是异或常数不同，可以相互验证。

C 语言实现使用 switch 还原八种情况。由于分支编号由 challenge 的最低 3 位决定，不同的 seed 会产生不同编号，因此可以通过不同 seed 测试各个分支，无需专门构造输入。

5. Phase 4

5.1 关键汇编分析

调用 p4_walk 前，程序先将搜索范围设为 [0,127]，目标值设为 challenge & 0x7f，再通过右移和异或计算附加参数。因此，函数调用可表示为 p4_walk(target, 0, 127, salt)。

p4_walk 的执行过程如下：

1)计算中点： 先计算上下界之差，再右移 1 位，最后加回下界，得到中点。这样可以避免直接将上下界相加可能产生的溢出。

2) 比较目标值与中点： 相等时直接返回；目标值较小时，在左半区间递归搜索；较大时，在右半区间递归搜索。

3) 计算返回值： 递归结束后，将递归返回值乘以 3，再与递归前算好的 (m+s) 异或，最后根据搜索方向异或常数 0x1a 或 0x0d，得到本层的返回值。

由此可见，p4_walk 不仅通过二分搜索确定目标值的位置，还会根据搜索路径逐层计算返回值。

5.2 恢复出的计算关系

目标值和附加参数分别为：

$$t = C \,\&\, 0x7f$$

$$s = (C \gg 7) \,\hat{}\, (C \gg 32)$$

递归过程在区间 $[l, h]$ 上进行，区间中点为：

$$m = l + \left\lfloor \frac{h-l}{2} \right\rfloor$$

定义递归函数 $F(t,l,h,s)$。

当目标值等于中点时，

$$F(t,l,h,s) = (m + 0xb8) \text{ ^ } s$$

当 t < m 时，先递归计算：

$$q = F(t,l,m-1,s)$$

再返回：

$$F(t,l,h,s) = (3q) \text{ ^ } (m+s) \text{ ^ } 0x1a$$

当 t > m 时，先递归计算：

$$q = F(t,m+1,h,s)$$

再返回：

$$F(t,l,h,s) = (3q) \text{ ^ } (m+s) \text{ ^ } 0x0d$$

这里的乘法与异或按 C 语言运算优先级解释，即先计算乘法，再执行异或；实际计算仍按 32 位无符号整数规则进行。初始区间为 [0,127]，最终答案为 $F(t,0,127,s)$。

5.3 推理过程

参数设置为搜索区间 [0,127] 和目标值 challenge & 0x7f，说明程序要在 128 个可能的值中查找目标，与递归二分搜索的判断一致。

递归层数按进入 p4_walk 的次数计算，包括首次调用。每次调用都会比较一次目标值与中点，因此调用次数也就是比较次数。目标值若恰好等于初始中点 63，只需 1 层；最深的搜索路径需要 8 层，仅出现在目标值为 127 时，区间依次为 [0,127]、[64,127]、[96,127]、[112,127]、[120,127]、[124,127]、[126,127]、[127,127]。对 0–127 的 128 个目标值逐一模拟后，层数为 k 的目标恰有 2^(k-1) 个（k=1,…,7），第 8 层只有目标 127 一个，各层目标数之和为 128。

递归返回值是本关的关键。找到目标时，函数直接返回 (m+0xb8) ^ s；未找到时，先递归搜索对应的半区间，再将子调用的返回值乘以 3，与本层在递归前算好的 (m+s) 异或，最后根据搜索方向异或 0x1a 或 0x0d。左右方向使用不同常数，因此可以据此核对分支是否写反。由于每一层都会重新计算并组合返回值，不能只根据目标值或搜索路径直接得出答案，而必须按实际的递归调用和返回顺序计算。调试时，可以使用 GDB 的 finish 逐层查看返回值，核对组合过程。

6. Phase 5

6.1 关键汇编分析

本关主要由 phase_05 和它调用的 p5_traverse 组成，分析重点如下：

1) 口令格式： esi=2 调用 valid_u32_fields，结合 sscanf 的解析结果，可确定口令由两个无符号整数构成。

2) 起始节点： mov 0x405b60(,%rax,8),%rdx 表示从地址 0x405b60 开始，按下标每隔 8 字节读取一个节点地址，由此确定起始节点由指针表查得。

3) 节点结构： mov (%rdx),%eax、mov 0x4(%rdx),%esi 和 mov 0x8(%rdx),%rdx 分别读取节点偏移 0x0、0x4 和 0x8 处的数据。结合访问间隔，可以判断节点依次存放 value、output 和指向下一节点的指针。

4) 状态更新： add (%rdx),%eax、lea (%rax,%rax,4),%eax 和 xor %esi,%eax 对应加上 value、乘以 5、再与 output 异或，构成状态更新的核心计算。

5) 保存输出： mov %esi,(%r8) 将当前节点的 output 写入指针参数指向的内存位置。

6) 遍历控制： and $0x3,%edi 将相关数值限制在 0–3，再通过 cmp %ecx,%edi 和 jae 控制循环何时结束。结合后继指针的读取，可以还原节点遍历的次数。

通过这些指令，可以逐步确定节点的内存布局、状态更新公式和遍历规则，再据此还原 p5_traverse 的计算过程。

6.2 恢复出的计算关系

根据汇编指令对节点的访问方式，可以恢复其结构。偏移 +0x0 处是 32 位整数 value，偏移 +0x4 处是 32 位整数 output，偏移 +0x8 处是 64 位后继指针 next。因此，每个节点占 16 字节，结构如下：

```
typedef struct {
   uint32_t value;  // +0x0
   uint32_t output;  // +0x4
   struct P5Node *next; // +0x8
} P5Node;
```

通过 objdump -s 查看地址 0x405ba0 至 0x405c50 的数据，并将后继指针转换为对应的节点索引，可以恢复完整的节点表。节点的 value、output 和后继节点索引如下：

节点索引	value	output	后继节点索引
0	0xf539	0x0550	4
1	0x751b	0x033b	11
2	0xb116	0x067f	0
3	0xaa1b	0x0a64	空指针
4	0x5a00	0x0004	9
5	0xa30c	0x072e	3
6	0x65bd	0x097d	1
7	0xf5e8	0x010c	2
8	0xd2f4	0x0879	5
9	0x935b	0x0416	8
10	0x426b	0x0b4c	6
11	0xebd0	0x022e	7

0x405b60 处按 8 字节间隔存放起始节点指针，起始节点表为：

challenge & 7	起始节点索引
0	10
1	6
2	1
3	11
4	7
5	2
6	0
7	4

令 c 为传入 p5_traverse 的 64 位 challenge。起始节点索引为：

$$i_0 = \text{start}[c\&7]$$

其中 start 为上述起始节点表。遍历控制量为：

$$n = (c \gg 3)\ \&\ 3$$

其取值范围为 0 至 3。程序首先处理起始节点，再沿后继指针继续访问，实际访问的节点总数为 $n+1$，最多访问 4 个节点。

初始状态为：

$$R_0 = (c \gg 6)\ \hat{}\ (c \gg 32)\ \hat{}\ 0xe284$$

其中参与后续运算的部分按 32 位无符号整数解释。设 v_k 为第 $k+1$ 个被访问节点的 value，w_k 为该节点的 output，则每处理一个节点，状态更新为：

$$R_{k+1} = [5(R_k + v_k)]\ \hat{}\ w_k \pmod{2^{32}}$$

遍历结束后，p5_traverse 返回最终的 32 位状态，输出变量中保存最后访问节点的 output。本关答案为两个整数：

$$(R_{\text{final}},\ w_{\text{last}})$$

其中，R_{final} 是按实际遍历路径计算得到的最终状态，w_{last} 是最后访问节点的 output。

6.3 推理过程

phase_05 将 challenge 保存到 r12。sscanf 解析出两个整数后，程序以 rdi=challenge、rsi=输出变量地址 调用 p5_traverse。函数返回值与第一个整数比较，输出变量中的值与第二个整数比较，因此可以确定两个口令的含义。

p5_traverse 首先用 challenge & 7 查起始节点表，确定第一个节点。初始状态由 challenge 的相关位经过异或运算和常数 0xe284 计算得到。每访问一个节点，程序依次执行：状态加上 value、乘以 5、再与 output 异或，同时将当前节点的 output 写入输出变量。状态运算按 32 位无符号整数进行，超出 32 位的部分会被舍弃。

遍历次数由 challenge 的相关位决定。当控制量为 0 时，程序直接返回；否则，计数器 ecx 从 1 开始，每轮通过 mov 0x8(%rdx),%rdx 读取后继指针并前进，再由 cmp 和 jae 判断是否结束。因此，实际访问节点数为控制量加 1，最多访问 4 个节点。

节点表通过 objdump -s 读取原始字节恢复。例如，节点 6 的 value 对应字节 bd 65 00 00，按小端序组合后为 0x65bd。每个节点占 16 字节，依次存放 4 字节的 value、4 字节的 output 和 8 字节的后继指针。将后继指针地址与节点基址 0x405ba0 对照，并按每个节点 16 字节的间隔换算为索引，即可得到 6.2 节中的节点表和起始节点表。

需要注意，节点 3 的后继字段在原始数据中确实是 64 位空指针，其所在位置 0x405bd0+8 的 8 个字节均为零。但从任何起点出发，程序最多访问 4 个节点，不会访问到节点 3，因此不会读取这个空指针。defuser.c 中的 p5_nodes 使用整数索引表示后继节点，并用 12 作为空指针的替代标记。这只是求解程序中的表示方式，并非原始数据；由于循环在最后一次访问后结束，该标记不会被用来访问节点。

因此，solve_phase_5() 的计算过程是：根据 challenge 查表确定起点，按实际遍历顺序逐节点更新状态，并记录最后一个节点的 output。最终得到的状态和最后一个 output，就是需要输入的两个整数。

7. Phase 6

7.1 关键汇编分析

phase_06 的入口与第 1 关类似，口令由一个整数构成，并与 p6_dispatch(challenge) 的返回值比较。因此，本关的关键是还原 p6_dispatch 的计算过程。

函数首先从地址 0x405e00 处的函数指针表中读取目标函数地址，并通过 call 调用。第一次调用使用的索引由 challenge 的相关位计算得到；第二次索引则通过 and $0x38 得到对应的字节偏移，再用于选择函数。这里需要注意，表中每个函数指针占 8 字节，因此字节偏移应除以 8 才能换算成表项索引。

两次调用之间，程序将第一次的返回值与 0x29b9 异或，并将结果作为第二次调用的参数。第二次调用结束后，再将返回值加上 0x2836，得到最终结果，也就是需要输入的整数。

7.2 恢复出的计算关系

定义六个基本运算函数：

$$f_0(x) = \mathrm{ROR}_{32}(x, 7)$$

$$f_1(x) = (x + 0xca8a) \text{ ^ } (x \gg 5)$$

$$f_2(x) = (4x + 0x1a62) \text{ ^ } (5x)$$

$$f_3(x) = ((x \gg 5) \text{ ^ } x) + 0x5f6c$$

$$f_4(x) = \mathrm{ROL}_{32}(x, 3) \text{ ^ } 0x44fe$$

$$f_5(x) = (x \text{ ^ } 0x4165) + 2x$$

其中，循环移位函数在 32 位范围内进行。

程序使用的函数表为：

$$\mathrm{ops} = [f_0, f_2, f_5, f_3, f_5, f_4, f_4, f_1]$$

定义两个索引：

$$i = C \,\&\, 7, j = (C \gg 3)\&7,$$

以及初始值：

$$a = (C \gg 6) \text{ ^ } (C \gg 32).$$

则程序首先计算：

$$u = \mathrm{ops}[i](a)$$

随后计算：

$$v = \mathrm{ops}[j](u \text{ ^ } 0x29b9)$$

最终答案为：

$$v + 0x2836$$

以上各式按 32 位无符号整数语义计算。

7.3 推理过程

首先，从 0x405e00 开始读取函数指针表中的 8 个地址，并逐个反汇编对应函数。对比后发现，8 个表项实际对应 6 种不同的函数，其中有两个函数各出现两次。将函数地址与表项位置对应起来，即可恢复 7.2 节中的 ops 表。

分析函数的运算方式时，有两个函数值得注意：一个实现右循环移位 rorl $7，另一个实现左循环移位 roll $3。循环移位会将移出一端的位补到另一端。其识别依据是“补位”移位对：x<<25 与 x>>7 的移位量之和恰为 32，左移补出的高位正是右移丢掉的低位，两者按位或即把溢出位绕回。另外，ror 7 与 rol 3 并不互逆：循环移位的逆是移位量相同的反向移位（ror a 的逆是 rol a），而移位量之和为 32 的两个循环移位（如 ror 7 与 rol 25）其实是同一个运算。其余四个函数则分别由加法、异或、移位和小常数乘法等操作组成，可根据对应指令逐一还原。

两次函数调用的索引分别由 challenge 的不同位计算得到，两个索引可能相同。即使两次选中同一个函数，也必须按原程序的顺序分别计算：先执行第一次调用，将返回值与 0x29b9 异或后作为第二次调用的参数，再对第二次返回值进行后续计算。

C 实现使用函数指针数组 ops 保留原程序的调用方式，并将 6 种函数分别实现，便于逐一对照反汇编结果。

8. Honor：Secret Phase

8.1 函数定位与关键汇编

首先，通过符号表定位相关函数：

```
nm -n ./bomb | grep -E 'secret_gate|secret_copy|secret_phase|solve_secret'
```

分析 secret_gate 的指令可知，程序先将栈上的 164 字节缓冲区清零，再调用 strlen 检查输入长度是否至少为 36 字节。随后，程序通过 secret_copy 将输入复制到缓冲区，并检查偏移 32 处的 4 个字节是否等于 0x4e45504f。

由于 x86-64 使用小端序，该常数在内存中的字节顺序为 4f 50 45 4e，对应 ASCII 字符串 OPEN。检查通过后，程序从缓冲区偏移 36 处开始读取内容，并将其与 challenge 和 context 一起传入 secret_phase。

因此，输入必须先包含 32 个空格，再写入 OPEN 和三个整数。

8.2 恢复出的计算关系

令 challenge 为 C，context 为 K。定义：

$$c_l = C \bmod 2^{32},\ c_h = \lfloor C/2^{32} \rfloor$$

$$k_l = K \bmod 2^{32},\ k_h = \lfloor K/2^{32} \rfloor$$

第一个整数为：

$$x = k_l \,\&\, 3$$

定义：

$$a = (c_l \wedge c_h) \wedge (k_l \wedge k_h)$$

汇编中的 rol 指令表示循环左移。定义：

$$\mathrm{ROL}_{32}(a, 1) = ((a \ll 1) | (a \gg 31)) \bmod 2^{32}$$

根据 secret_mix 中的指令，得到：

$$m = (\mathrm{ROL}_{32}(a, 1) + 0xfcb) \wedge 0x782b$$

同时定义：

$$b = (x \cdot 0xfcb) \wedge (K \gg 16)$$

于是第二个整数为：

$$y = m \wedge b$$

第三个整数为：

$$z = y \wedge [(k_l \wedge k_h) + x + 0x782b + ((c_l \wedge c_h) \gg 1)]$$

以上运算按 32 位无符号整数规则执行。这里的右移为逻辑右移。

8.3 推理过程

输入格式的依据已在 8.1 节说明，接下来根据 secret_phase 中的三次比较，依次还原三个整数的计算方式。

1) 第一个整数： 第一次比较发生在其他计算之前，只涉及 context 的低 2 位，因此第一个整数由 context & 3 确定。

2) 第二个整数： 第二次比较将 secret_mix 的返回值与一个计算结果进行异或，再与第二个整数比较。secret_mix 依次执行左循环移位 1 位、加 0xfcb、异或 0x782b。此外，另一条计算链通过乘以 0xfcb，再与 K >> 16 异或得到中间结果。常数 0xfcb 同时出现在两处，可以用来核对指令与公式是否对应。

3) 第三个整数： 第三次比较前，lea 0x782b(%r8,%rcx,1) 将两个寄存器的值与常数 0x782b 相加。随后，程序再进行加法和异或运算，得到第三个整数。分析时应按照指令的执行顺序还原公式，避免遗漏中间步骤。

solve_secret() 使用 "%32s" 配合空字符串生成 32 个前导空格，使 OPEN 正好位于缓冲区偏移 32 处，再在其后输出计算得到的三个整数，组成完整的 Secret Phase 输入。

9. 测试与结果记录

9.1 每个关卡的测试截图

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab2/ics-2026-bomblab$ python3 public_test.py --phase 1
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
```

phase1

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab2/ics-2026-bomblab$ python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
```

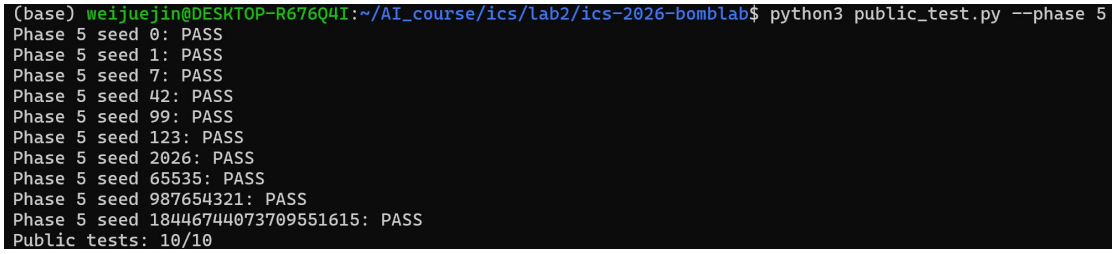
phase2

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab2/ics-2026-bomblab$ python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

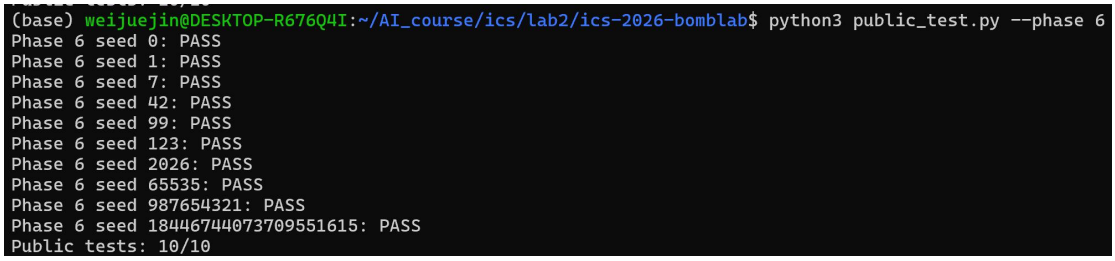
phase3



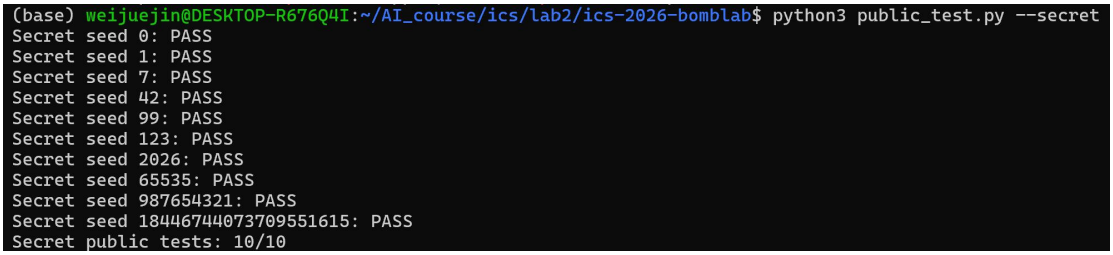
phase4



phase5



phase6



honor

除截图外，python3 public_test.py --secret 的公开测试结果为 **Secret public tests: 10/10**，覆盖 seed 0、1、7、42、99、123、2026、65535、987654321 以及 64 位无符号整数的最大值 18446744073709551615。第 8 节与第 9 节分别提供两种证据：前者是推导证据，说明公式如何由汇编指令恢复；后者是测试证据，说明多组输入下炸弹判定与公式输出一致。公开测试只覆盖有限个 seed，不等于对所有输入的穷举证明，隐藏测试能否通过仍取决于公式本身的正确性。

9.2 公开列表以外的 seed 1205 验证

以下为 seed 1205 下六关与 Honor 的逐关验证截图（./defuser --seed 1205 … | ./bomb --seed 1205 …）：

```
(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab2/ics-2026-bomblab$ { ./defuser --seed 1205; ./defuser --secret --seed 1205; } | ./bomb --seed 1
205
──────────────────────────────────────────────────────────────
BOMBLAB 2026                                    这里还没有明天

                          *
                           \
                        .-"""-.
                      .'       '.
                     /    LOOP   \
                    |             |
                     \           /
                      '---------'

[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

──────────────────────────────────────────────────────────────

一个普通的晚上，你和林夏在实验室接上了一台旧设备，
却误入了一座陌生的数字城市。住在这里的，是 AI。
街边的维修铺还开着，老板正低头摆弄一盏灯。
再往前走，维护站的门半开着，里面只有一台旧终端。
屏幕上留着一条消息：

  没想到这里还会有人来。
  祝你们好运。
  — sky

窗口角落留着‘返回实验室’的按钮。
林夏把外套搭在椅背上：‘先看看？’
维护面板上画着一枚炸弹，旁边亮着六项未完成的交接检查。
它不是街上的爆炸物，只是旧系统给这次维护选的图标。
```

```
第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0xeac4c37dbf38df2a
Input (unsigned decimal integers):
Phase 1 defused.

门锁响了一声，维护站开始接收新的记录。
日志里的请求时间各不相同，城市的日期却始终是同一天。
其中一份校时记录来自街那头的钟楼。林夏记下地址，推开门。

[1 已解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0xa4b8295da4f7753a
Input (unsigned decimal integers):
Phase 2 defused.

校时检查通过了。钟楼送出一条新记录，几秒后又收回原样的一条。
‘不是钟没走。’林夏把两条记录放在一起，‘它送出去又回来了。’
你们沿着线路往下查，重复的地址最后指向街口的分流站。

[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]
```

```
第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x9f0e982350f12fbc
Input (unsigned decimal integers):
Phase 3 defused.

这次请求没有再绕回来。分流站把它送进档案馆，收到一条回复：
‘未找到交接存档。’回复后面附着一份旧目录。
林夏收好路线图。你们穿过马路，档案馆的门刚刚打开。

[1 已解除] [2 已解除] [3 已解除] [4 当前] [5 待解除] [6 待解除]

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x6a11c60f99f8f8e4
Input (unsigned decimal integers):
Phase 4 defused.

最后一层索引返回了一份完整存档，还有那次迁移留下的记录。
新设备接上时，交接没有做完。城市只能反复恢复最后一份旧存档；
居民今天做过的事，到了夜里就都没了。
存档后面列着几处还没核对的设备。你认出了维修铺那盏灯。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 当前] [6 待解除]
```

```
第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xbf1ec6d520879e06
Input (unsigned decimal integers):
Phase 5 defused.

最后一处连接也对上了，今天的设备状态第一次进入待保存的记录。
维修铺老板试着拧了一下灯座，灯亮了一瞬，又被他小心地关掉。
‘留到明天装好。’他说。
交接清单上只剩最后一项。你们带着记录走到回廊尽头。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 当前]

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x5e17ce7298155208
Input (unsigned decimal integers):
Phase 6 defused.

终端逐项读过记录，最后停在‘保存完成’。
林夏没有马上起身。你们等了一会儿，屏幕上的日期向前走了一天。
这次没有恢复旧存档。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。
```

```
Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。

阅读器亮起来。最先出现的不是代码，是两份很早的留言。

  许遥：

  北方那边的通知到了，我准备去。
  那个项目我想试很久了。你帮我看申请的时候，我还说不一定选得上，
  现在真能去了，反而有点不习惯。以后遇到问题，
  不能再把椅子滑到你旁边，问一句‘你看这是不是有毛病’。

  你想留下继续手里的事，我记得你为它准备了多久。
  我也想看看，最后能做成什么样。
  这里没做完的几处，我把记录整理好了。
  你有空就改，没空先忙自己的事。

  走之前出去吃顿饭吧。这次别在实验室凑合了。

  —— sky

隔了几天，许遥在后面添了一份回复。

  吃饭的时间你定。

  这里的东西我会留着。你在北方有了新想法，也可以回来添。
  我暂时不走，这边的事刚有起色，想再试一阵。
  以后说不定会改主意，到时候告诉你。你也一样。
  别因为申请时说得很坚定，遇到不喜欢的事就不好意思讲。

  第一场雪记得拍。我想看看你一直说的北方。

  —— 许遥

后面是这些年断断续续的更新。
一张北方的雪照，一次换了方向的项目，一段拖了很久才修好的代码。
sky 去了北方，许遥留在这里。他们各自忙起来，
回复有时隔了好几天，也会聊新认识的朋友。
有一阵，许遥没再继续原来的项目。后来 sky 也换了工作。
这座城倒还在。谁回来，就顺手添一点东西。

观测台的灯是 sky 修的，两把椅子是许遥后来搬来的。
最后一张便签贴在灯座旁边：

  ‘下次回来，先坐一会儿。’

林夏拉开其中一把椅子。你们坐了一会儿，没有再翻记录。
街上亮起今天的新灯。

离开观测台时，你想起自己收藏了很久的一份项目介绍，
把它放进明天的待办。林夏也在自己的本子上记了几行。

退出前，终端收到 sky 的消息：
‘日志看到了，新存档正常。谢谢你们，剩下的维护我来接。’

实验室的灯还亮着，外套仍搭在椅背上。你确认文件都保存了，合上电脑。
林夏揉了揉眼睛：‘还活着，精神状态另说。’
你把椅子推回桌下：‘事已至此，先睡觉吧。’
你们关掉灯，走出了实验室。

──────────────────────────────────────────────────────────────

谢谢你远道而来，与我相遇。

晚安。

—— sky

──────────────────────────────────────────────────────────────

(base) weijuejin@DESKTOP-R676Q4I:~/AI_course/ics/lab2/ics-2026-bomblab$
```

10. 实验总结

本实验的主要难点有两个。其一是理清数据的传递与计算过程：challenge 被拆分为哪些位段、.rodata 中的表格如何被访问，以及递归和链表的返回值如何逐步组合。为此，我导出了入口函数和辅助函数的反汇编代码，沿着寄存器中的数据逐条追踪，并用原始字节核对表格内容。例如，bd 65 00 00 按小端序还原为 0x65bd；对于跳转表，则逐项核对“表项—跳转桩—函数体”的对应关系。其二是确保还原出的计算关系对不同输入都成立，而不只是适用于调试时使用的某个 seed。为此，我在断点处核对关键计算结果，并结合公开测试和额外的 seed 进行验证。

完成六关与 Honor 后，本人最大的体会是：循环、分支和递归等整体结构通常比较容易辨认，真正容易出错的往往是具体的运算细节，例如 lea 中的比例变址如何实现乘法、移位与补位如何实现循环移位、整数运算如何按 (2^{32}) 回绕，以及函数的符号名与实际跳转关系可能不一致（如 p3_case_N 并不一定对应第 N 个分支）。整体结构帮助确定分析方向，而这些细节决定了还原出的公式能否与二进制代码严格一致。测试记录见第 9 节。

11. 参考资料

1. 课程提供的 ICS Lab 2 / BombLab 实验说明及测试脚本。
2. 课程提供的 bomb 可执行文件与相关源文件。
3. GNU Binutils 文档：objdump 和 nm 的使用说明。
4. GNU GDB 文档：程序调试、寄存器检查和内存检查。
5. GCC 文档：C 语言编译选项及整数类型相关说明。

12. 辅助 AI 工具使用说明

12.1 使用对象

本实验使用了 Deepseek、豆包、zcode 工具协助完成。

12.2 使用详情说明

1) 依据 ICS 课程提供的实验文档，将本实验常用的指令提取出来。

2) 对编写好的 C 语言代码进行 debug。

3) 反汇编卡壳的时候提供思路（如 Phase 3 的跳转表映射）。

4) 使用 zcode 批量核对反汇编（比对各辅助函数的常量与运算）、整理报告文字与排版。报告中的公式、数据与结论均以自行导出的反汇编和第 9 节的测试结果为准，未将工具输出的推导直接当作结论。